A dataset contains four samples that are given below. Each sample is a sequence of words.

- I enjoyed the movie Transformers — 5
- We live among its people now — 6
- They have much to learn — 5
- Freedom is the right of all sentient beings — 8

The sequences have no continuity (that is, changing the order of the sequences does not matter). Let the embedding dimension of each word be $\mathbb{R}^{1\times 256}$.

1) Suppose we want the transformer to process each sequence (in its entirety). What should be the minimum context length (window) T?

Minimum Context length → 8

1. **Sequence Processing:**
   - Transformer models process sequences of tokens (words or subwords) and generate representations for these tokens.
   - Each token in the sequence is embedded into a vector space, and these embeddings are then processed by the model.

2. **Context Length:**
   - The context length (T) refers to the maximum number of tokens that the model can process in a single forward pass.
   - This is crucial because it determines how many tokens from the input sequence can be considered simultaneously by the self-attention mechanism.

Determining Minimum Context Length

To ensure that each sequence can be processed entirely without truncation, you need to set the context length (T) to at least the length of the longest sequence in your dataset.

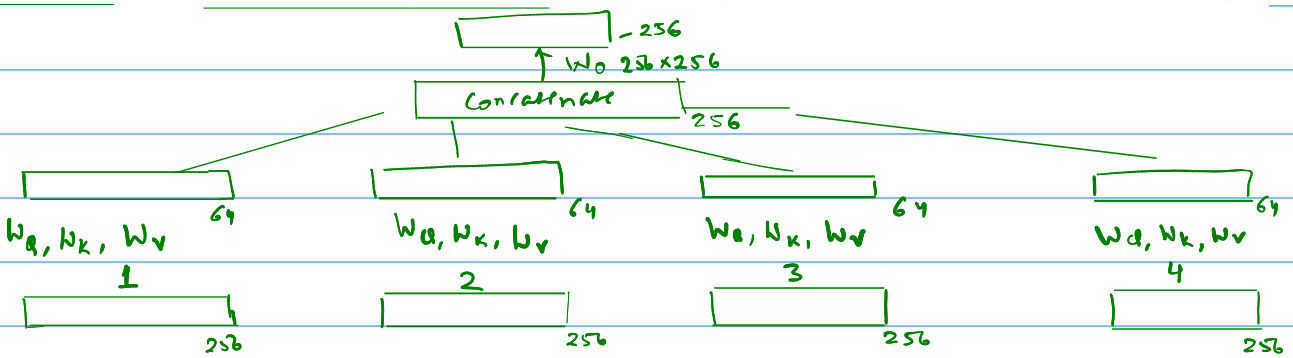
Steps to Determine Minimum Context Length:

1. **Identify Sequence Lengths:**
   - Count the number of words (or tokens) in each sequence.
2. **Find the Longest Sequence:**
   - Determine which sequence has the most words.
3. **Set Context Length:**
   - Set (T) to be at least as long as this longest sequence.

2) Suppose we pack all the sequences into a batch and pass it to the encoder of the transformer. The dimension of the projection matrices $(W_Q, W_K, W_V) \in \mathbb{R}^{256\times 64}$. Then, which of the following tensors denotes the correct dimension of the input to the encoder

- ○ $4 \times 256 \times 8$
- ◉ $4 \times 8 \times 256$ (Batch Size, Token Size, embedding dimension.)
- ○ $8 \times 256 \times 64$
- ○ $4 \times 256 \times 64$

3) Suppose we use four attention heads and an additional linear mapping $W_O \in \mathbb{R}^{256\times 256}$. Then, how many parameters are there in the multi-head attention sub-layer?



No. of P

W_Q → 256×64

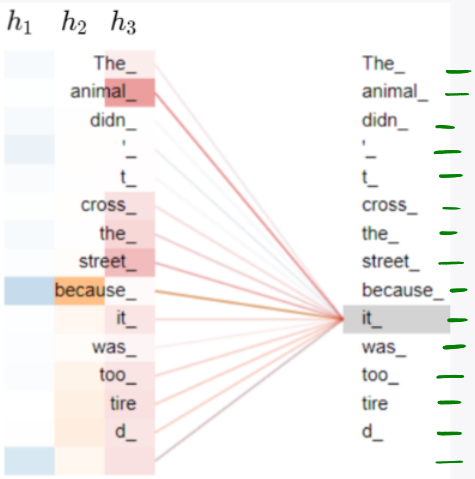
W_K → 256 × 64 × 4 + 256 × 256.

W_V → 256×64

$3 \times 256 \times 64 \times 4 + 256 \times 256 = 256 \times 256 (3 + 1)$

$= 256 \times 256 \times 4 = 262144$

The diagram below shows the attention scores, from three independent heads (h_1, h_2, h_3), for the word ``it_".



15

Each attention head is denoted by a colour (say, $h_3 : red$) and the attention scores computed in the head are represented by its shades (for h_3, shades of red). The darker the colour, the higher the attention score. The last row represents the attention score for a special token. Suppose the size of the attention matrix in $i-th$ head is $\mathrm{A}_i \in \mathbb{R}^{m \times n}$. Assume zero-based indexing where required.

4) What is the value of m? and n? Attention Matrix has be 15 x 15.

$\therefore m = 15$
$n = 15$

The input embeddings for the words ``learning", ``brings" and ``joy" are $h_1 = [0.5, 0.25, 1]$, $h_2 = [0.1, 0.25, 0]$, and $h_3 = [0.1, 0.1, 0.9]$, respectively. Note that the embeddings are row vectors. The projection matrices are as follows $W_Q = \begin{bmatrix} 1 & 1 \\ -1 & 1 \\ 0 & 1 \end{bmatrix}$ $W_K = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$ $W_V = \begin{bmatrix} 0 & 0 \\ -1 & -1 \\ 1 & 1 \end{bmatrix}$ Let z_j denote the linear combination of the value vectors for the $j-th$ word.

row vector

3×2 3×2 3×2

7) While constructing z_1, more attention was given to the word ``learning". The statement is

- (●) True
- ( ) False

$h_1 = [0.5, 0.25, 1]_{1\times3}$, $W_Q = \begin{bmatrix} 1 & 1 \\ -1 & 1 \\ 0 & 1 \end{bmatrix}_{3\times2}$

$Q_1 = h_1 W_Q = \begin{bmatrix} 0.5-0.25+0 \\ 0.5+0.25+1 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 1.75 \end{bmatrix}_{2\times1}$

$K_1 = h_1 W_K \begin{bmatrix} 0.25 \\ 1.5 \end{bmatrix}_{2\times1}$ and $V_1 = \begin{bmatrix} 0.75 \\ 0.75 \end{bmatrix}_{2\times1}$

α (attention weights) $=$ Softmax $\left[\frac{Q^T \cdot K}{\sqrt{d_Q}}\right]$ $d_Q = 2$

$\therefore \alpha_1 = \left[\text{Softmax}\left(\frac{Q_1^T \cdot K_1}{\sqrt{2}}\right), \text{Softmax}\left(\frac{Q_1^T \cdot K_2}{\sqrt{2}}\right), \text{Softmax}\left(\frac{Q_1^T \cdot K_3}{\sqrt{2}}\right)\right]$

using numpy $\alpha_1 = [0.58, 0.10, 0.32]$

$\therefore$ more attention on 'learning' — first word.

8) While constructing z_2, all words were given equal attention

- ( ) True
- (●) False

for z_2, $\alpha_2 = \left[\text{softmax}\left(\frac{Q_2^T \cdot K_1}{\sqrt{K_d}}\right), \text{Softmax}\left(\frac{Q_2^T \cdot K_2}{\sqrt{2}}\right), \text{Softmax}\left(\frac{Q_2^T \cdot K_3}{\sqrt{2}}\right)\right]$

using numpy, $\alpha_2 = [0.38, 0.27, 0.35]$

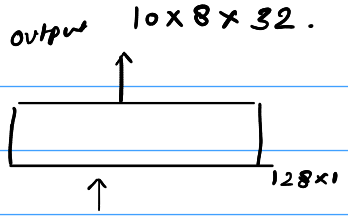
$\therefore$ different attention to every word.

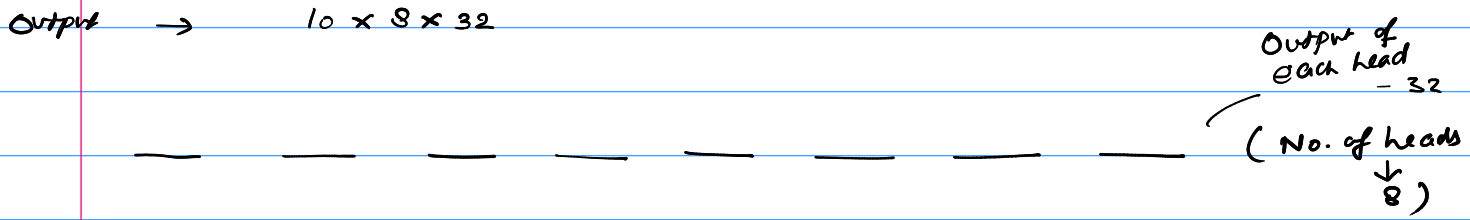
$\therefore$

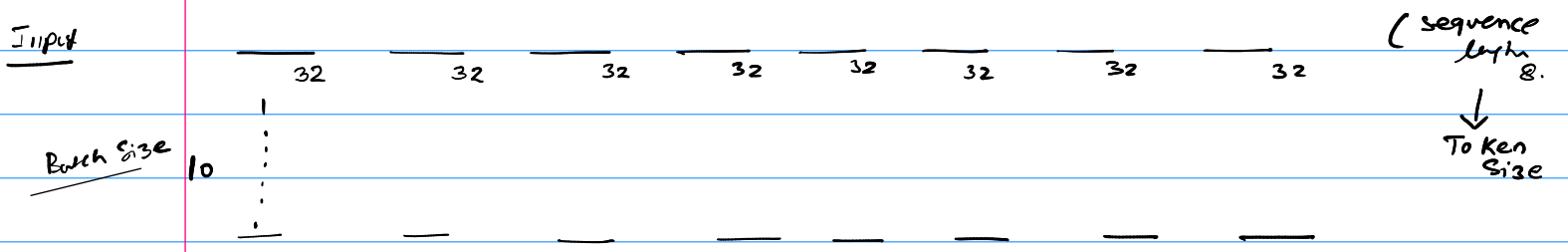
•

Imp. terms & Notations

```
{'input': {'batch_size': 10, 'embed_dim': 32, 'seq_len': 8, 'vocab_size': 10},
 'model': {'d_ff': 128,      → dimension of fully connected layer.
           'd_model': 32,    → output dimension
           'dk': 4,          → key
           'dq': 4,          → query
           'dv': 4,          → value
           'n_heads': 8,     → no. of heads
           'n_layers': 6}}   → no. of layers.
```







∴ input = 10 x 8 x 32

Inside One layer → 2 Sub layers < multi head self attention layer
fully connected Forward neural network.

Multi Head Self attention layer. (for this example)

$d_model = 32$. no. of heads = 8.

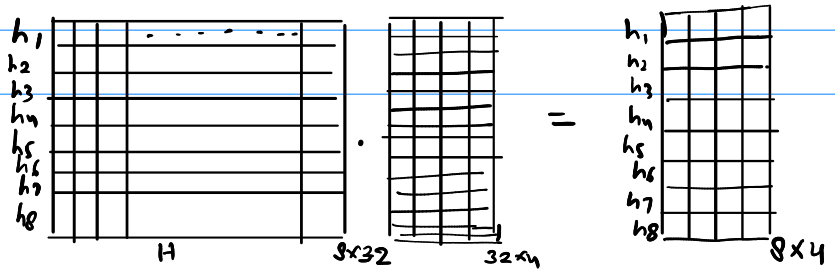
$\therefore d_K : d_Q : d_V : \frac{32}{8} = 4$

One head.

Input is 8x32 (H), W_Q, W_K & W_V of size $d_{model} \times d_K \rightarrow 32 \times 4$

$Q = H.W_Q$ = 8x4 size, same for K and V.

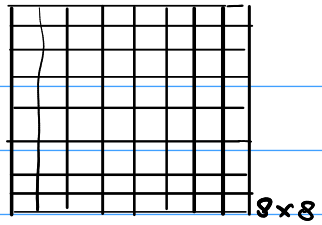
So.



→ Q, K and V sizes. 8x4

$\alpha = \text{Softmax}\left(\frac{Q K^T}{\sqrt{k_d}}\right)$, $Q \rightarrow 8 \times 4$, $K^T \rightarrow 4 \times 8$.

$\therefore \alpha$ size will be 8×8

$\alpha \rightarrow$ 

and attention = $\alpha . V$ = $(8 \times 8) . (8 \times 4)$ = 8×4

$\therefore$ output from one head = 8×4

Concatenate output from 8 heads $\rightarrow$ 8×32

Multi head attention = $\underbrace{\text{Concat}(\alpha_i V_i)}_{8 \times 32} . W_O$ = $(8 \times 32) . W(32 \times 32)$

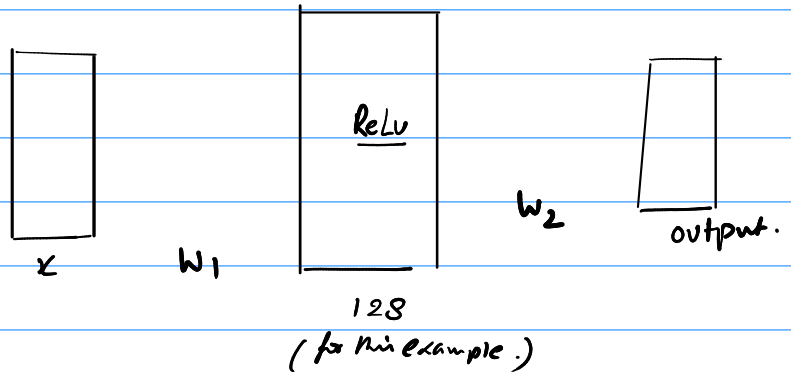
= 8×32

W_O Size $\rightarrow$ no. of heads $\times d_v \times d_{model}$

= 32×32

$\therefore$ from multi-head attention sub layer, we get output of 8×32 size and for 10 batches $10 \times 8 \times 32$.

Now we pass it through a simple Feed forward Neural Network

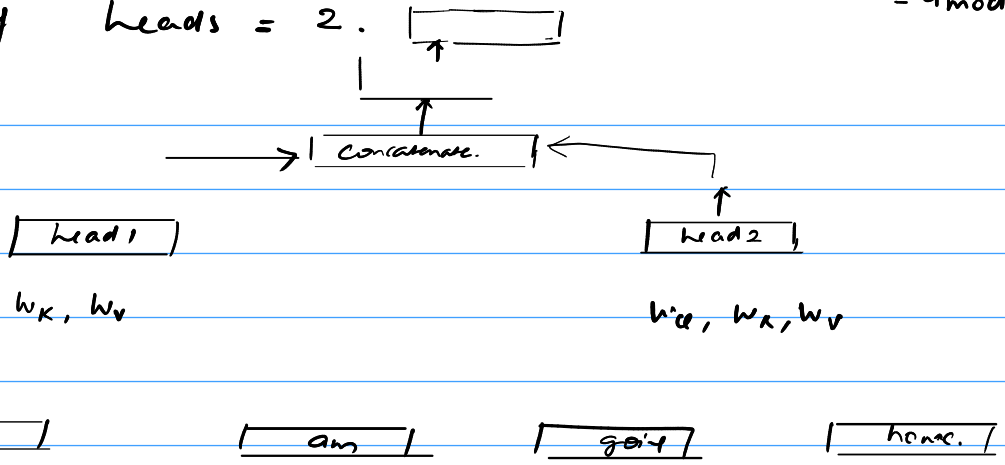


$\therefore W_1 = 32 \times 128$

output = $\text{Max}(0, xW_1 + b_1) W_2 + b_2$.

Simple example Sequence length → 4, embedding size → 4 = d_{model}

No. of heads = 2.



Input size → 4 × 4

$d_K = \frac{d_{model}}{\text{No. of heads}} = \frac{4}{2} = 2.$

∴ W_Q, W_K, W_V of size 4 × 2

∴ $Q = H \cdot W_Q = (4 \times 4) \cdot (4 \times 2) = 4 \times 2$

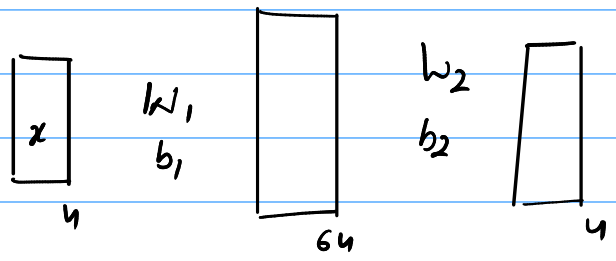
$\alpha = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{2}}\right) =$ 4 × 4 size.

Multi-head attention = Concat $(\alpha \cdot V) \cdot W_0$, $W_0 \in$ No. of heads × d_v × d_{model} = 4 × 4

= (4 × 4) · (4 × 4) = 4 × 4

∴ input (4 × 4) output (4 × 4)

Now pass through forward neural network of size 64. (one by one)



output = $\max(0, x W_1 + b_1) \cdot W_2 + b_2$. (Size 4 × 4)

Lets Say - 6 layers.

Parameters → In one layer 2 heads. W_Q, W_K, W_V $\frac{4 \times 2 \times 2}{16}$

W_0 — 4 × 4 → 16

W_1 — 4 × 64 → 256

b_1 — 64 → 64

W_2 → 64 × 4 → 256

b_2 → 4 → 4

∴ Total no. of parameters = 16 + 16 + 256 + 64 + 256 + 4 = 96 + 516 = 612

```
{'input': {'batch_size': 10, 'embed_dim': 32, 'seq_len': 8, 'vocab_size': 10},
 'model': {'d_ff': 128,
           'd_model': 32,
           'dk': 4,
           'dq': 4,
           'dv': 4,
           'n_heads': 8,
           'n_layers': 6}}
```

Lets check no. of Parameters for 2 layers, including embedding layer.

Embedding layer → 10 × 32
vocab_size × embed_dim.

One layer Multi head = 32 × 32 × 3
= 1024 × 3 = 3072

W_Q, W_K, W_V Size 32 × 32

W_O Size 32 × 32 (projection layer)

∴ no. 'p' in W_O = 1024

∴ in multi head Sub-layer = 3072 + 1024 = 4096

Now FFN Sublayer W_1, W_2 32 × 128 + 128 × 32
= 8192

∴ Tot 'p' in 1 encoder layer = 4096 + 8192 = 12288
+ layer norm → 64 ∴ Tot → 12352

Now 2 layers → 12352 × 2 = 24704

Now output layer = 32 × 10 = 320

∴ Tot no of Parameters = 24704 + 320 + 320
= 25344

Imp to remember

✓ embedding layers also gets updated.

✓ nn. Parameter.

nn. Linear

1. `nn.Embedding` selects the rows of the given matrix, given a list of integers
2. `nn.Linear` does the einsum operation `...d, d e -> ...e`
3. `nn.Parameter` basically just makes a tensor trainable (receive gradients and updates on step). this is the lowest level you can go, so actually, you can define your entire deep neural network with just nn.Parameters and manually do all the above with gathers and matrix multiplies or einsums

For the PA. : Parameters.

embedding layer ⟶ $10 \times 32 \rightarrow 320$

(No bias in nn. Embedding())

output layer → Linear layer, with bias

$P = 32 \times 10 + 10 = 330.$

Encoder layer

MHA ⟶ $3 \times 32 \times 32$ (W_Q, W_K, W_V)

and 32×32 (W_O)

∴ Total → $4 \times 32 \times 32 \rightarrow$ 4096

FFN. $128 \times 32 + 32 \times 128 \rightarrow$ 8192 ✓

Normalisation layers → mha_layer → $32 + 32 = 64$

ffn_layer → $32 + 32 = 64$

∴ Total $P = 8192 + 4096 + 330 + 320 + 64 + 64$

$= 13066$

320 — Embedding layer @ 10 × 32

1024 — W_Q 32 × 32

1024 → W_K 〃

1024 → W_V 〃

1024 → W_O 〃

32 | → MHA_normalization_layer.

32 |

4096 → W_1 → 32 × 128

4096 → W_2 → 128 × 32

32] — FFN normalization layer

32]

320 → W_L 32 × 10

10 → bias of W_L.

∴ Total → 13066

Now with 2 layers

Embedding → 320

output layer → 330.

2 layers = (8192 + 4096 + 64 + 64) × 2

= 12416 × 2 = 24832

∴ Total = 24832 + 320 + 330

= 25482

320 — Embedding layer.

1024
1024
1024
1024
32
32
} MHA1

4096
4096
32
32
} FFN1

1024
1024
1024
1024
32
32
} MHA2

4096
4096
32
32
} FFN2

320
10
} output layer.

Total = 320 + (4096 + 64 + 8192 + 64) × 2 + 330

= 25482

Points to remember

nn. Embedding does not have bias

nn. Parameter does not have bias

nn. Linear has "bias" internally

bias = False not set

default bias = True.

1. I am going to Delhi tomorrow (6)

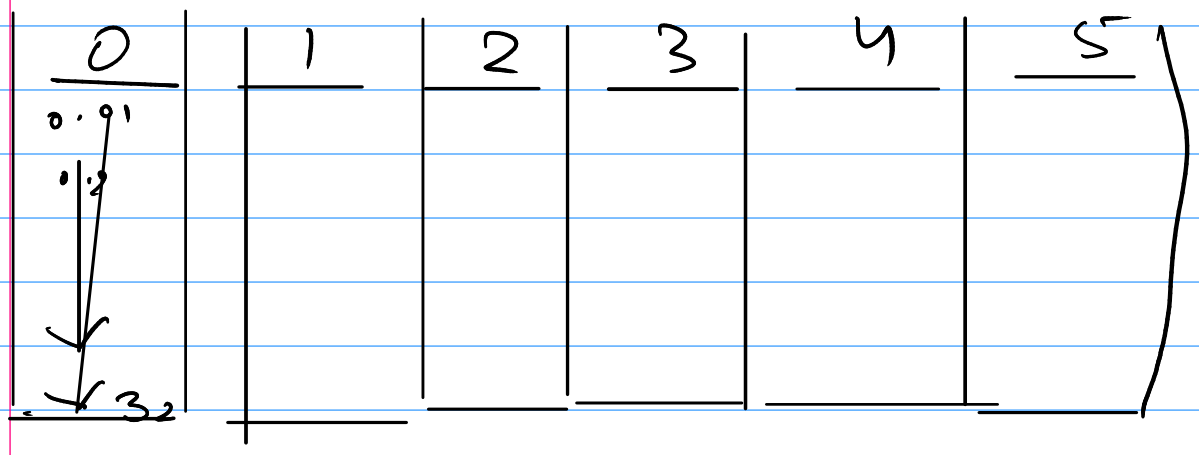
2. Delhi is my home.

3. I am going to my home.

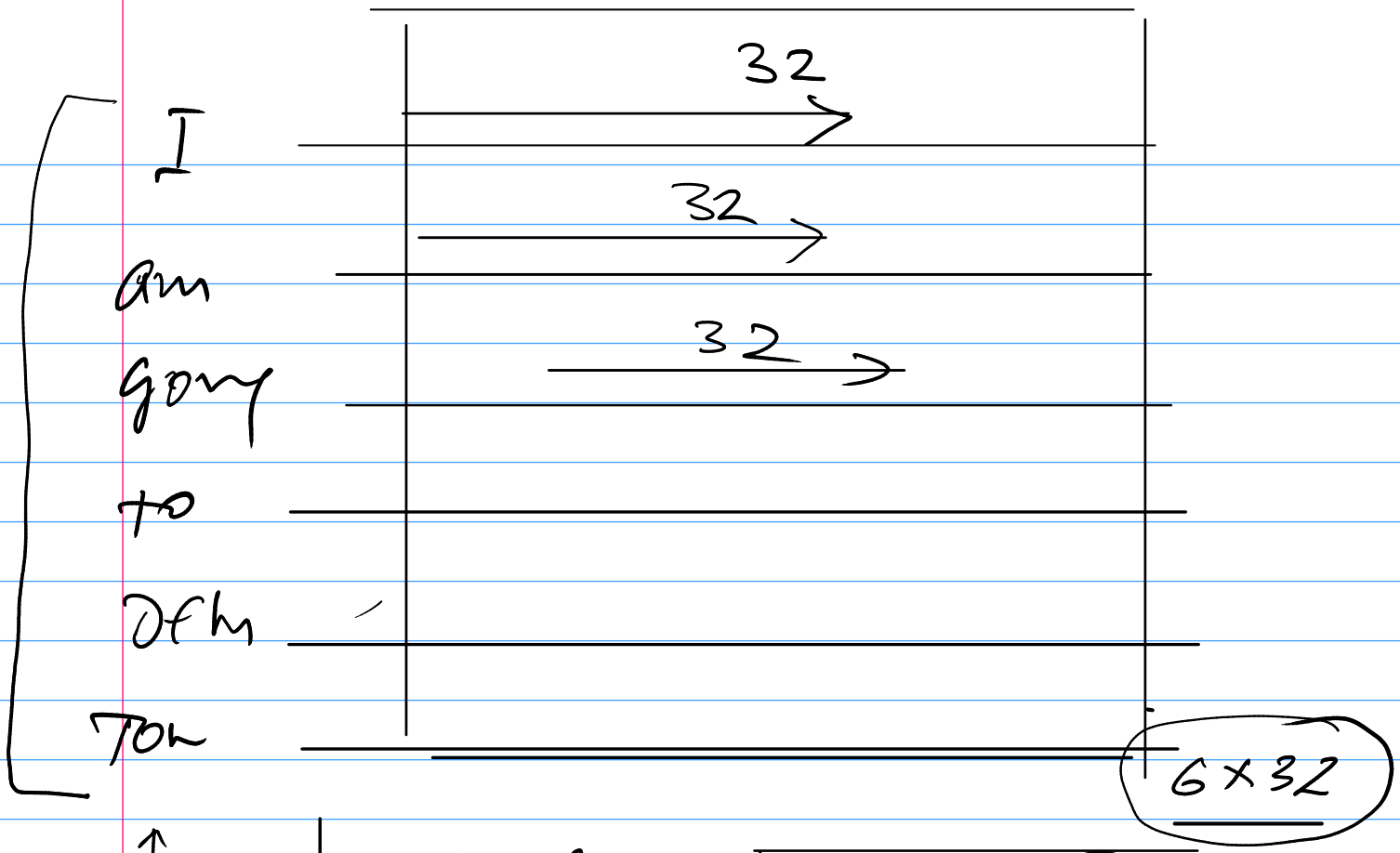
Vocab = [I, am, going, to, Delhi, tomor, is, my, home] 9.

	1	2	3	4	5
I	am	going	to	Delhi	Tomo
Delhi	is	my	home		0
0	1	2	3	7	8
I	am	going	to	my	home

6.



nn. Embedding (,)

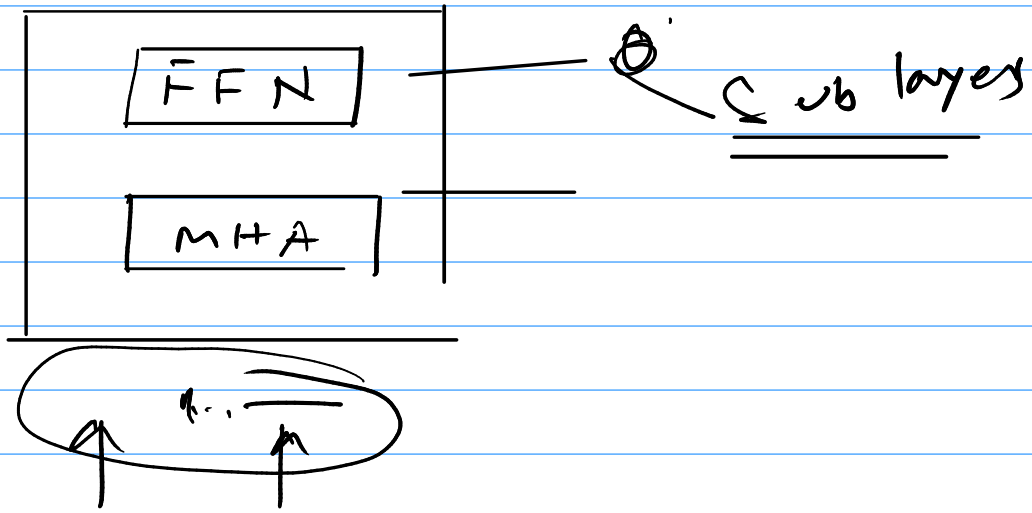


batch size

10 × 6 × 32

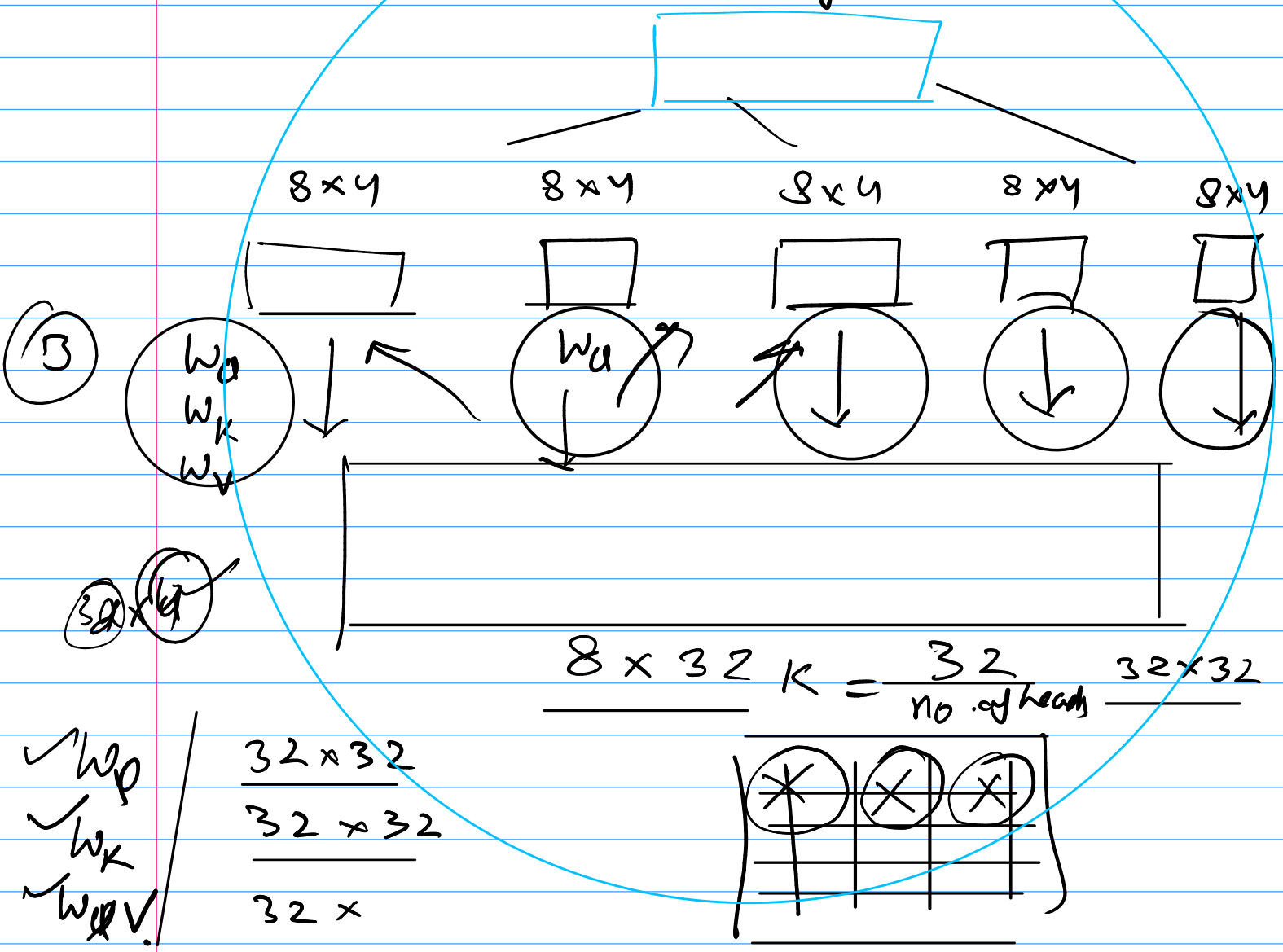
positional encoding + embedding layer

over.



Seq_length = 8, emb_dim = 32

no. of heads = 8.



W_Q | 32 x 32

W_K | 32 x 32

W_V | 32 x

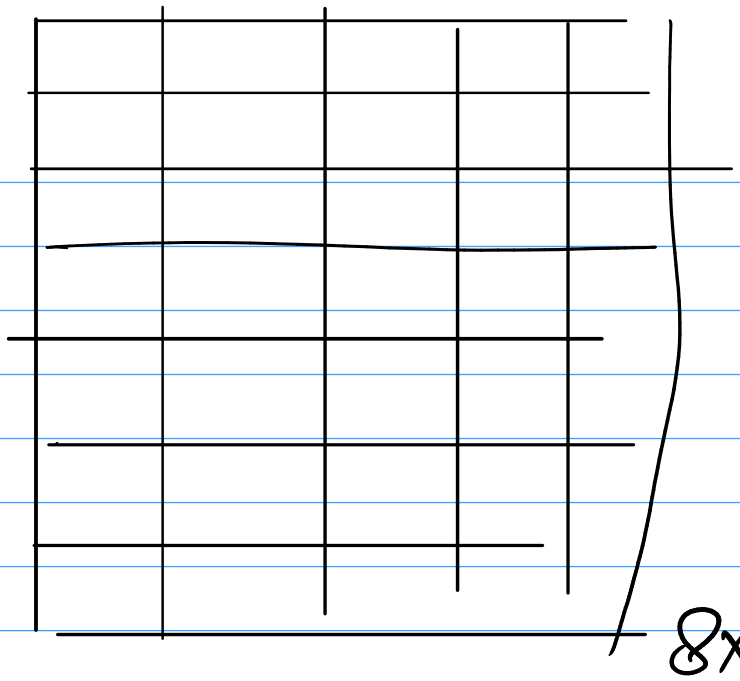
W_Q, W_K, W_V = 32 x 32

Q = H x W_Q = 8 x 32 x 8 x 32

Q, K, & V. → 8 x 32

Attention_weights = Softmax $\left(\frac{Q \cdot K^T}{\sqrt{K_d}} \right)$

8 x 32 x 32 x 8

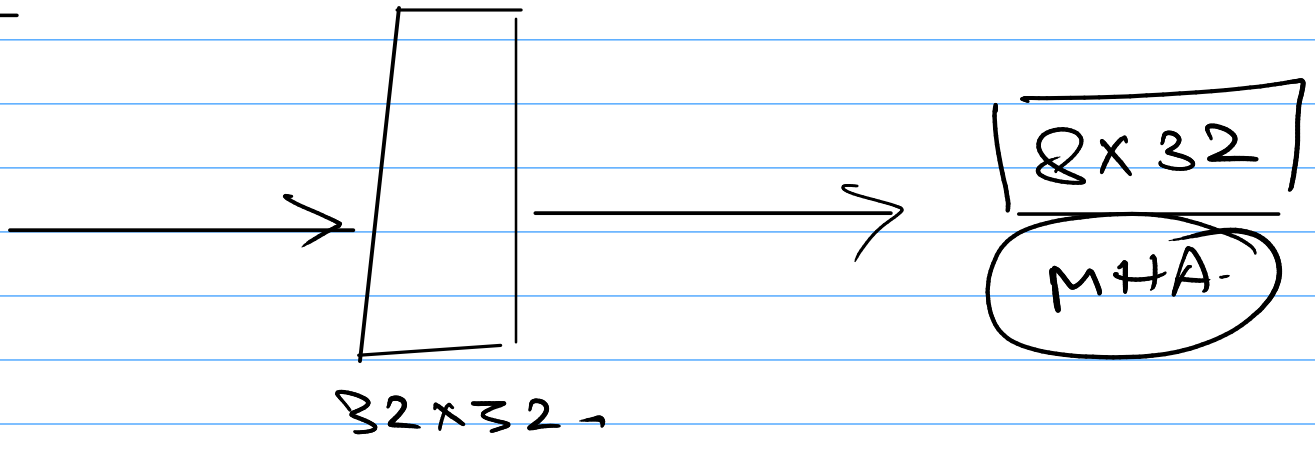


x = Attn weight . V.

$8 \times 8 \times 8 \times 32$

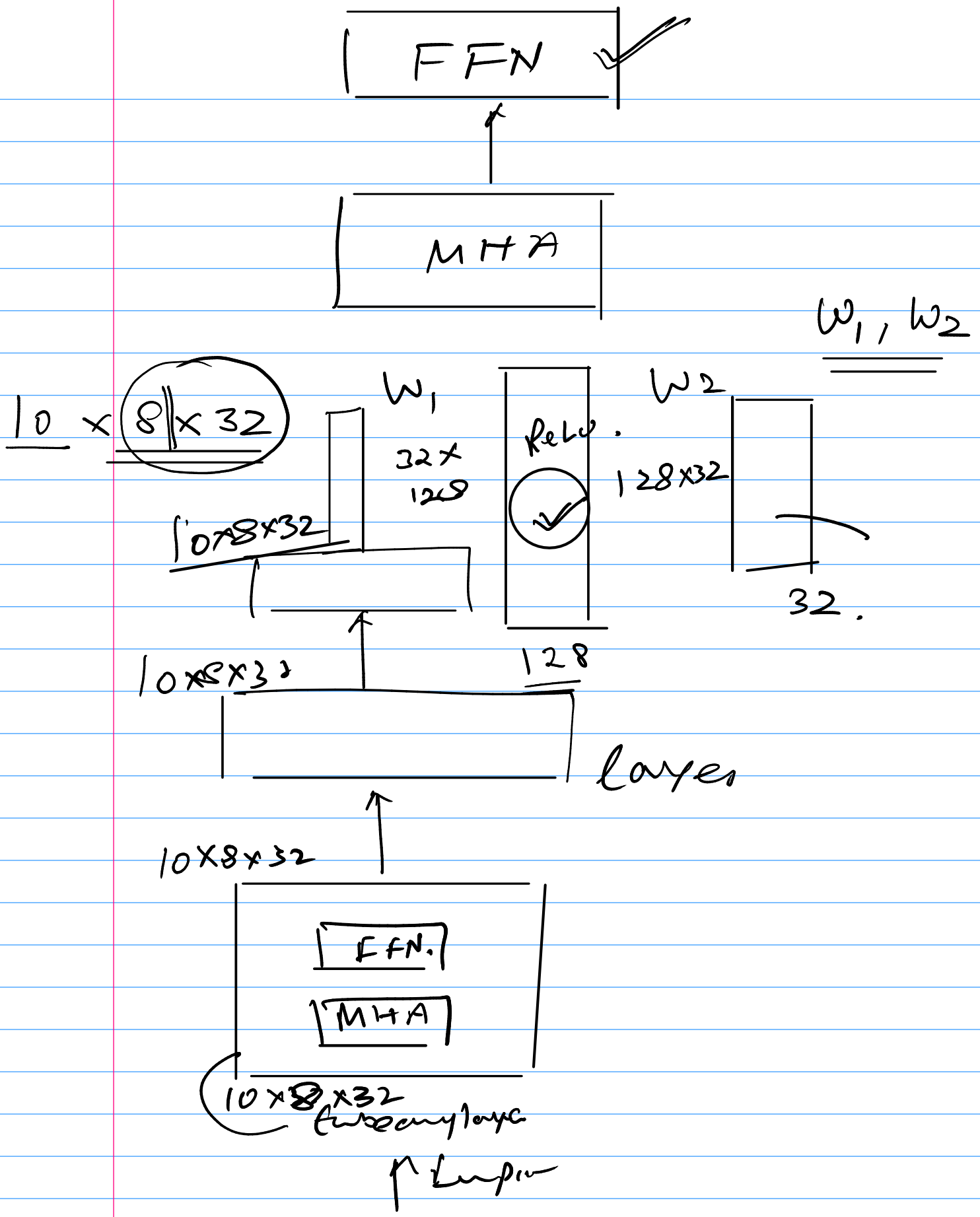
$= \boxed{8 \times 32}$

W_O



Embedding layer

$W_Q, W_K, W_V, W_O.$ || → out (MHA)


FFN
MHA
W_1, W_2
10 x 8 x 32
W_1
32 x 128
ReLU.
W_2
128 x 32
10x8x32
32.
128
10x8x32
layer
10x8x32
FFN.
MHA
10x8x32
Encoder layer

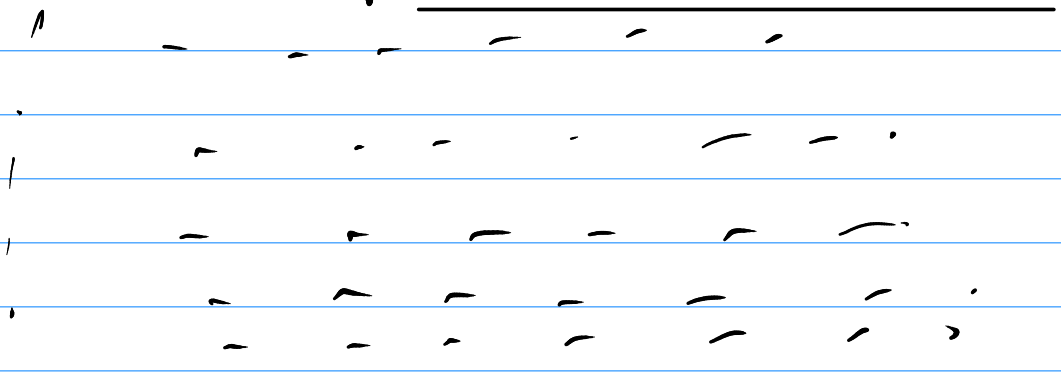
10 x 8 x 9

↑

output layer.

10 x 8 x 32

Vocab size → 9.



I	argmax 0.9	0.15	0.2	0.05	0.1	0.7	0.80.
am							
going							
home							
No							
Idea							
Tomorrow							

Class MHA.

W_q
W_k
W_v
W_o

def forward

Class FFN.

W_1

W_2.

def forward.

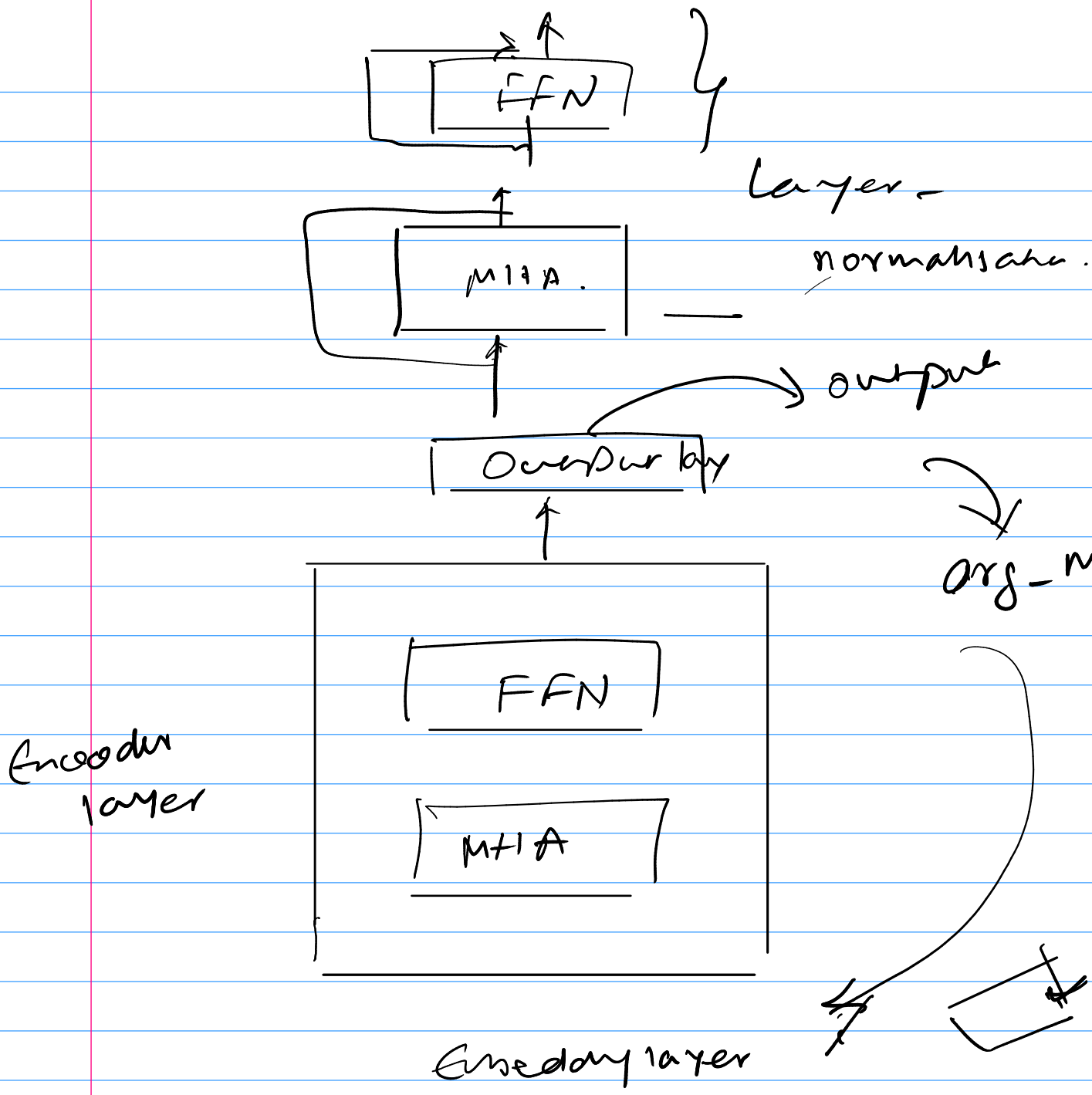
Class Encoder layer

def forward.

def output layer. W_L.

for

Class def Encoder


FFN
layer-
normalisation.
MHA.
output
OverDur lay
arg_max.
FFN
MHA
Encoder
layer
Encoder layer